

# Organizzazione del Codice

---

## Organizzazione del Codice

*Come dividere un programma C++ in più file usando header file e file di implementazione.*

## Organizzare il codice in più file

Quando scrivi un programma piccolo, un unico file `.cpp` va benissimo. Ma immagina di dover scrivere un gestionale scolastico con centinaia di funzioni: tutto in un file solo sarebbe un disastro. Trovare una funzione richiederebbe minuti.

Dividere il codice in più file risolve questo problema. Ogni file ha uno scopo preciso, come i capitoli di un libro.

Vantaggi:

- **Leggibilità:** ogni file è focalizzato su un argomento
- **Riutilizzo:** puoi usare le stesse funzioni in più programmi
- **Manutenzione:** modifichi una parte senza toccare il resto
- **Collaborazione:** più persone lavorano su file diversi senza interferirsi

## I due tipi di file

In C++ il codice viene diviso in due tipi di file:

| Tipo            | Estensione                          | Cosa contiene                         |
|-----------------|-------------------------------------|---------------------------------------|
| Header          | <code>.h</code> o <code>.hpp</code> | Le <b>dichiarazioni</b> (cosa esiste) |
| Implementazione | <code>.cpp</code>                   | Le <b>definizioni</b> (come funziona) |

Il file header è come il **sommario di un libro**: ti dice cosa c'è, ma non ti mostra tutto il contenuto. Il file `.cpp` è il contenuto vero e proprio.

## Esempio pratico

Immaginiamo di creare una piccola libreria con funzioni matematiche personalizzate.

## Struttura dei file

```
progetto/  
├─ main.cpp  
├─ matematica.h  
└─ matematica.cpp
```

### Il file header: **matematica.h**

Contiene solo le **firme** delle funzioni (senza il corpo):

```
// matematica.h – le dichiarazioni  
  
#ifndef MATEMATICA_H    // protezione dall'inclusione multipla (spiegata più  
sotto)  
#define MATEMATICA_H  
  
int somma(int a, int b);  
int prodotto(int a, int b);  
double potenza(double base, int esponente);  
  
#endif
```

### Il file di implementazione: **matematica.cpp**

Contiene il codice vero delle funzioni:

```
// matematica.cpp – le implementazioni

#include "matematica.h" // include il proprio header

int somma(int a, int b) {
    return a + b;
}

int prodotto(int a, int b) {
    return a * b;
}

double potenza(double base, int esponente) {
    double risultato = 1.0;
    for (int i = 0; i < esponente; i++) {
        risultato *= base;
    }
    return risultato;
}
```

## Il file principale: `main.cpp`

Include l'header per sapere che le funzioni esistono:

```
// main.cpp
#include <iostream>
#include "matematica.h" // include il nostro header (virgolette, non <>)
using namespace std;

int main() {
    cout << "Somma: " << somma(3, 4) << endl;
    cout << "Prodotto: " << prodotto(3, 4) << endl;
    cout << "2^8 = " << potenza(2, 8) << endl;
    return 0;
}
```

Per compilare, passa **tutti** i file `.cpp` al compilatore:

```
g++ main.cpp matematica.cpp -o programma
./programma
```

## Le include guard: protezione dall'inclusione multipla

Nell'header hai visto queste righe "strane":

```
#ifndef MATEMATICA_H
#define MATEMATICA_H

// ... contenuto ...

#endif
```

Si chiamano **include guard**. Servono a evitare un errore comune: se lo stesso header venisse incluso due volte nello stesso file, il compilatore si lamenterebbe di funzioni "ridefinite".

Come funzionano:

1. `#ifndef MATEMATICA_H` — "se MATEMATICA\_H non è ancora definito..."
2. `#define MATEMATICA_H` — "...definiscilo..."
3. "...ed esegui tutto questo contenuto"
4. `#endif` — fine del blocco

La seconda volta che il file viene incluso, `MATEMATICA_H` è già definito, quindi tutto il contenuto viene saltato automaticamente.

## Alternativa moderna: `#pragma once`

```
#pragma once // includi questo file una sola volta

// contenuto dell'header...
```

`#pragma once` fa la stessa cosa delle include guard, ma è più semplice da scrivere. È supportata da tutti i compilatori moderni.

# Come includere file

Ci sono due sintassi per `#include` :

```
#include <iostream>    // parentesi angolari → librerie di sistema
#include "matematica.h" // virgolette → file del tuo progetto
```

## Esempio con una classe

Le classi si dividono esattamente nello stesso modo: la dichiarazione nell'header, i metodi nel `.cpp` .

### studente.h

```
#pragma once

#include <string>
using namespace std;

class Studente {
private:
    string nome;
    int eta;
    double votoMedio;

public:
    Studente(string n, int e);           // solo le firme dei metodi
    void aggiungiVoto(double voto);
    void stampaInfo() const;
    string getNome() const;
    double getVotoMedio() const;
};
```

## studente.cpp

```
#include "studente.h"
#include <iostream>
using namespace std;

// L'implementazione dei metodi usa NomeClasse:: davanti al nome del metodo
Studente::Studente(string n, int e) {
    nome = n;
    eta = e;
    votoMedio = 0.0;
}

void Studente::aggiungiVoto(double voto) {
    if (votoMedio == 0.0) {
        votoMedio = voto;
    } else {
        votoMedio = (votoMedio + voto) / 2.0;
    }
}

void Studente::stampaInfo() const {
    cout << "Nome: " << nome << ", Eta: " << eta
        << ", Media: " << votoMedio << endl;
}

string Studente::getNome() const { return nome; }
double Studente::getVotoMedio() const { return votoMedio; }
```

## main.cpp

```
#include <iostream>
#include "studente.h"
using namespace std;

int main() {
    Studente s("Mario", 16);
    s.aggiungiVoto(8.5);
    s.aggiungiVoto(7.0);
    s.aggiungiVoto(9.0);
    s.stampaInfo();
    return 0;
}
```

Compilazione:

```
g++ main.cpp studente.cpp -o programma
```

## Cosa va dove

Nell'header ( `.h` ) metti:

- Le firme delle funzioni (prototipi)
- Le dichiarazioni delle classi
- Le definizioni di `struct` ed `enum`
- Le costanti ( `const` )

Nel file `.cpp` metti:

- Il corpo delle funzioni
- Il corpo dei metodi delle classi
- La logica del programma

**Regola pratica:** se qualcuno ha bisogno di *sapere che esiste*, mettilo nell'header. Se qualcuno ha bisogno di *vedere come funziona*, mettilo nel `.cpp`.

## Errori comuni

```
// Dimenticare le include guard → errori di ridefinizione
// Soluzione: aggiungi #pragma once o #ifndef

// Includere il .cpp invece dell'header
#include "matematica.cpp" // SBAGLIATO
#include "matematica.h"   // CORRETTO

// Dimenticare di compilare tutti i file .cpp
g++ main.cpp -o programma // SBAGLIATO – manca matematica.cpp
g++ main.cpp matematica.cpp -o programma // CORRETTO
```